

UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingeniería de Sistemas
Informáticos



UNIVERSIDAD
POLITÉCNICA
DE MADRID



Máster
Deep Learning
Universidad Politécnica de Madrid

Trabajo Fin de Máster

Sistema LLM para análisis financiero histórico
mediante generación automática de código

MÁSTER DE FORMACIÓN PERMANENTE
EN DEEP LEARNING

Presentado por:

Carlos Ruiz Oyarzun

Bajo la dirección de:

Adrián Girón Jiménez

Alejandro Delgado Peribáñez

Madrid, 2026

Resumen

Este trabajo presenta el diseño, implementación y evaluación de un sistema multi-agente basado en modelos de lenguaje para realizar análisis financiero histórico a partir de consultas en lenguaje natural. El objetivo del proyecto no es predecir el mercado ni recomendar inversiones, sino estudiar si un LLM puede integrarse en un flujo más controlado que transforme una petición abierta en una respuesta sustentada por datos y cálculos observables.

El sistema desarrollado organiza el proceso en varias etapas: resolución y validación de datos históricos, planificación del análisis, generación de código Python, validación previa a la ejecución, ejecución controlada e interpretación final. Como aportación principal, la arquitectura incorpora trazabilidad completa y mecanismos de control que permiten conservar artefactos intermedios, detectar fallos por fase y acotar las reparaciones cuando aparecen errores. La evaluación se apoya en pruebas automatizadas y en una batería de consultas financieras de distinta dificultad. En conjunto, los resultados muestran la viabilidad de una arquitectura LLM más explicable, revisable y robusta que una respuesta generativa directa de una sola etapa.

Palabras clave: análisis financiero histórico, modelos de lenguaje, generación de código, agentes LLM, ejecución controlada, trazabilidad.

Abstract

This dissertation presents the design, implementation and evaluation of a multi-agent language-model-based system for historical financial analysis from natural-language queries. The aim is not to predict market behaviour or provide investment advice, but to study whether an LLM can be integrated into a controlled workflow that turns an open request into a response grounded in observable data and calculations.

The implemented system structures the process into several stages: historical data resolution and validation, analysis planning, Python code generation, pre-execution validation, controlled execution and final interpretation. Its main contribution lies in the combination of full traceability and explicit control mechanisms, which make it possible to preserve intermediate artefacts, identify failures by stage and bound repair attempts when errors occur. The evaluation relies on automated tests and on a graded set of financial-analysis queries with different levels of difficulty. Overall, the results support the feasibility of an LLM-based architecture that is more explainable, reviewable and robust than a single-step direct generative response.

Keywords:

historical financial analysis, language models, code generation, LLM agents, traceability.

Índice general

Resumen	I
Abstract	II
Abreviaturas y acrónimos	VIII
1. Introducción	1
1.1. Contexto y definición del problema	1
1.2. Motivación	2
1.3. Objetivos	2
1.4. Alcance	3
1.5. Estructura de la memoria	3
2. Estado de la cuestión	5
2.1. Enfoque de la revisión	5
2.2. Modelos de lenguaje y agentes financieros	5
2.3. Agentes, herramientas y orquestación	6
2.4. Generación y ejecución controlada de código	7
2.5. Fiabilidad y evaluación de sistemas generativos	7
2.6. Comparativa y posicionamiento de la propuesta	8
3. Material de trabajo y planteamiento experimental	10
3.1. Origen y naturaleza del material de trabajo	10
3.2. Casos y escenario experimental de partida	11
4. Metodología	13
4.1. Enfoque metodológico	13
4.2. Pregunta de investigación y objetivo evaluable	13
4.3. Unidad de análisis	14
4.4. Diseño experimental	14
4.5. Protocolo general de ejecución	15
4.6. Criterio general de éxito	15

4.7. Evaluación de casos no completados	15
4.8. Evaluación de casos completados	16
4.9. Escala de puntuación y rúbrica	17
4.10. Revisión humana	18
4.11. Amenazas a la validez	18
5. Flujo multiagente del sistema	20
5.1. Visión general del sistema	20
5.2. Recorrido completo de una ejecución	22
5.2.1. Fase 1. Resolución y validación de datos	22
5.2.2. Fase 2. Planificación analítica y generación de código	25
5.2.3. Fase 3. Validación y ejecución	26
5.2.4. Fase 4. Interpretación final	27
5.3. Mecanismos de corrección y criterios de parada	28
5.4. Prompts como contratos operativos	30
5.4.1. Prompts de sistema compartidos	30
5.4.2. Prompts de la fase de datos	30
5.4.3. Prompts de planificación, generación, validación y reparación . .	31
5.4.4. Prompt del interpretador	33
6. Resultados y evaluacion	34
6.1. Funcion de este capitulo	34
6.2. Enfoque recomendado	34
6.3. Preguntas que convendra responder mas adelante	34
6.4. Plantilla de estructura futura	35
6.4.1. Resultado global	35
6.4.2. Lectura por grupos de casos	35
6.4.3. Analisis por etapas	35
6.4.4. Casos representativos	35
6.4.5. Discusion critica	35
7. Limitaciones, aspectos éticos y reproducibilidad	36
7.1. Limitaciones del estudio	36
7.2. Aspectos éticos y uso responsable	37
7.3. Reproducibilidad	38
8. Conclusiones y trabajo futuro	39
8.1. Funcion de este capitulo	39
8.2. Guia para las conclusiones	39
8.3. Guia para el trabajo futuro	39

A. Anexos	43
A.1. Ejecucion del sistema	43
A.1.1. Preparacion minima	43
A.1.2. Ejecucion con ejemplos integrados	43
A.1.3. Ejecucion con una entrada JSON propia	44
A.1.4. Salida y artefactos generados	44
A.2. Configuracion LLM y API	45
A.2.1. Configuracion recomendada	45
A.2.2. Variables equivalentes admitidas	45
A.2.3. Parametros de generacion	46
A.2.4. Comprobaciones practicas	46
A.3. Notas para reproducibilidad	46

Índice de tablas

2.1. Comparación entre los trabajos relacionados y la propuesta desarrollada.	8
5.1. Rutas de reparación incorporadas en el workflow.	29

Índice de figuras

5.1. Flujo multiagente de referencia para el desarrollo del sistema de análisis financiero.	21
---	----

Abreviaturas y acrónimos

API Application Programming Interface.

CSV Comma-Separated Values.

ETF Exchange-Traded Fund.

LLM Large Language Model.

OHLCV Open, High, Low, Close, Adjusted Close y Volume.

RAG Retrieval-Augmented Generation.

UTC Coordinated Universal Time.

Capítulo 1

Introducción

1.1. Contexto y definición del problema

Los modelos de lenguaje de gran escala han ampliado de forma notable las posibilidades de interacción con los sistemas informáticos. Además de producir texto, pueden interpretar instrucciones, proponer procedimientos y generar código ejecutable. Buena parte de este avance se apoya en la arquitectura Transformer y en el mecanismo de atención, que constituyen una de las bases de los modelos de lenguaje actuales.

La capacidad de generar código resulta especialmente interesante en tareas de análisis de datos. Una consulta expresada en lenguaje natural puede transformarse en operaciones de carga, filtrado, cálculo y presentación de resultados. Sin embargo, esta flexibilidad introduce un problema de confianza: que el código sea plausible no significa que sea correcto, y que una respuesta esté bien redactada no garantiza que respete los datos de partida.

Este problema cobra especial importancia en el ámbito financiero. Incluso cuando el análisis se limita a información histórica, una cifra incorrecta o una interpretación poco prudente puede conducir a conclusiones equivocadas. Por ello, no basta con pedir al modelo que responda directamente a una consulta: también es necesario conocer qué datos ha utilizado, qué métricas ha calculado, qué código ha ejecutado y qué limitaciones presenta el resultado.

Este trabajo aborda el problema desde la integración de sistemas, no desde el entrenamiento de un nuevo modelo. Para ello, se desarrolla un prototipo que recibe una consulta financiera en lenguaje natural, resuelve y descarga los datos históricos necesarios, organiza el análisis mediante varios agentes LLM y mantiene los cálculos dentro de un flujo programático controlado. El sistema separa la fase de datos, la planificación analítica, la generación y validación de código, la ejecución y la interpretación final, y conserva artefactos que permiten reconstruir cada ejecución completa.

1.2. Motivación

El proyecto nace del interés por combinar tres ámbitos que a menudo se estudian por separado: los modelos de lenguaje, la programación para el análisis de datos y la interpretación financiera. Los LLM permiten construir con rapidez interfaces capaces de entender peticiones diversas, pero esa misma facilidad puede ocultar errores cuando todo el proceso queda reducido a una única respuesta generativa.

La motivación principal es comprobar hasta qué punto puede aprovecharse la flexibilidad del modelo sin renunciar a mecanismos claros de control. En el sistema propuesto, el LLM no actúa como una caja negra que responde directamente, sino como parte de un flujo en el que primero se resuelven los datos, después se planifica el análisis, más tarde se genera y valida el código, y solo al final se redacta la respuesta. Esta separación permite revisar tanto el planteamiento seguido como los cálculos realmente obtenidos.

Otro criterio importante es que los fallos sean visibles. Si la petición de datos no puede resolverse con fiabilidad, si la descarga no devuelve información útil, si el código generado no es aceptable o si la ejecución no produce una salida válida, el sistema registra el problema y devuelve un resultado controlado. Este comportamiento resulta preferible a completar una respuesta con cifras no verificadas. La evaluación mediante una batería de consultas de distinta dificultad permite observar estos fallos e identificar en qué parte del proceso se producen.

El prototipo se construyó de forma incremental, desde consultas sencillas hasta una arquitectura multiagente con validaciones intermedias, reparación acotada y trazabilidad completa. La idea de fondo de este trabajo es que, en un dominio sensible como el financiero, la utilidad de un sistema basado en LLM depende no solo de que responda, sino de que pueda explicarse cómo se ha llegado a esa respuesta.

1.3. Objetivos

El objetivo general del trabajo es diseñar, implementar y evaluar un sistema basado en un modelo de lenguaje capaz de transformar consultas sobre datos financieros históricos en cálculos ejecutables y respuestas trazables.

Los objetivos específicos son:

- Diseñar una arquitectura modular que separe la resolución de datos, la planificación del análisis, la generación y validación de código, la ejecución de los cálculos y la elaboración de la respuesta final.
- Desarrollar un flujo capaz de interpretar consultas con diferentes requisitos analíticos y determinar de forma explícita qué datos, métricas y operaciones necesita cada caso.

- Resolver, validar y descargar datos históricos de mercado de forma controlada antes de pasar a la fase analítica.
- Generar y ejecutar código Python adaptado a cada consulta, aplicando controles que limiten errores de estructura, de ejecución y de formato de salida.
- Garantizar la trazabilidad del proceso mediante la conservación de contratos intermedios, código generado, resultados estructurados, artefactos de ejecución y registros de error.
- Generar respuestas financieras basadas exclusivamente en los cálculos realizados, sin incorporar predicciones ni recomendaciones de inversión.
- Evaluar la capacidad del sistema para comprender la consulta, seleccionar métricas pertinentes, ejecutar el análisis de forma correcta y producir respuestas claras, prudentes y fundamentadas.

1.4. Alcance

El sistema se limita al análisis histórico y descriptivo de activos financieros. Trabaja con datos de mercado obtenidos dentro del propio flujo y reutiliza también artefactos persistidos para pruebas, trazas y evaluación. Los casos utilizados incluyen acciones, índices, fondos cotizados, divisas, una materia prima y un criptoactivo, con diferentes periodos e intervalos temporales.

El prototipo no obtiene precios en tiempo real durante la ejecución, no incorpora noticias ni información macroeconómica y tampoco realiza análisis fundamental. No pretende predecir movimientos futuros ni recomendar la compra o venta de activos. Estas restricciones responden al propósito del trabajo: evaluar una arquitectura LLM trazable, no construir un asesor financiero.

1.5. Estructura de la memoria

La memoria se organiza en torno a tres ejes principales: la metodología, el flujo multiagente y la evaluación del sistema. El Capítulo 2 revisa los trabajos previos y sitúa la propuesta dentro del estado de la cuestión. El Capítulo 3 resume el material de trabajo y el planteamiento experimental de partida.

El Capítulo 4 define la metodología con la que se estudiará el sistema: unidad de análisis, protocolo de ejecución, criterios de evaluación y amenazas a la validez. El Capítulo 5 se reserva para el flujo multiagente del sistema y para la explicación detallada de sus etapas, contratos, validaciones y rutas de recuperación. El Capítulo 6 recogerá los resultados y la evaluación final del proyecto.

Los últimos capítulos se plantean de forma más compacta. El Capítulo 7 reunirá las limitaciones, los aspectos éticos y las condiciones de reproducibilidad, mientras que el Capítulo 8 cerrará la memoria con las conclusiones y el trabajo futuro. Por último, los anexos conservarán solo la información operativa de apoyo que siga siendo útil para ejecutar el sistema, documentar su configuración LLM y describir los artefactos técnicos que deja cada ejecución.

Capítulo 2

Estado de la cuestión

2.1. Enfoque de la revisión

Este capítulo sitúa el trabajo dentro del uso de modelos de lenguaje para el análisis financiero y la automatización de tareas de datos. No se pretende revisar todo el campo de la inteligencia artificial aplicada a las finanzas, sino centrarse en las líneas que sustentan el sistema desarrollado: la especialización de modelos en el dominio financiero, las arquitecturas basadas en agentes, el uso de herramientas externas, la generación de código como artefacto intermedio y la evaluación de sistemas generativos [13, 15].

El punto de partida del proyecto es un modelo abierto de propósito general, `openai/gpt-oss-20b`, integrado en un flujo de trabajo controlado [11]. Ante una consulta en lenguaje natural, el modelo participa en la planificación del análisis, la generación de código Python y la redacción de una respuesta apoyada en datos históricos locales. Quedan fuera del alcance el entrenamiento de un modelo propio, la predicción de precios y la toma autónoma de decisiones de inversión.

2.2. Modelos de lenguaje y agentes financieros

La aplicación del procesamiento del lenguaje natural al ámbito financiero es anterior a los modelos generativos actuales. FinBERT adapta BERT al lenguaje financiero y muestra la utilidad de los modelos especializados en tareas como el análisis de sentimiento [1]. Posteriormente, BloombergGPT y FinGPT ampliaron esta línea mediante modelos entrenados o ajustados con noticias, documentación corporativa y otros recursos propios del sector [16, 19].

Estos trabajos muestran que el lenguaje financiero incorpora terminología, relaciones y matices que conviene tratar con cuidado. Sin embargo, también sugieren que la especialización del modelo no es la única vía posible. En este trabajo se adopta una estrategia complementaria: se emplea un modelo generalista de pesos abiertos dentro

de una arquitectura que restringe entradas, valida artefactos y ancla la respuesta en datos observables [11]. El interés, por tanto, no está en comparar modelos aislados, sino en estudiar el valor de su integración en un sistema verificable de análisis histórico.

La evolución reciente de los LLM financieros también ha favorecido la aparición de sistemas basados en agentes. FinRobot propone una arquitectura orientada a la investigación y valoración de empresas, mientras que TradingAgents distribuye tareas de análisis, debate y toma de decisiones de *trading* entre varios agentes [17, 21]. En ambos casos, el modelo deja de actuar solo como generador de texto y pasa a intervenir en un proceso compuesto por distintas responsabilidades. El proyecto comparte esa idea de separación funcional, aunque con un objetivo más acotado: analizar datos históricos de forma trazable, sin operar sobre el mercado ni recomendar inversiones.

2.3. Agentes, herramientas y orquestación

Los sistemas basados en agentes permiten que un modelo vaya más allá de la generación de texto: puede utilizar herramientas y trabajar con los resultados de acciones externas. ReAct propone alternar razonamiento y actuación para que el modelo observe las consecuencias de sus decisiones [20]. Toolformer estudia cómo un modelo puede aprender a recurrir a herramientas cuando las necesita [14]. Las revisiones sobre agentes basados en LLM describen esta evolución como el paso de sistemas puramente conversacionales a arquitecturas capaces de planificar, actuar y aprovechar observaciones del entorno [15].

Para coordinar estas tareas es necesario conservar la información producida durante la ejecución. En la práctica, esto exige mantener un estado compartido, definir etapas con responsabilidades separadas y establecer transiciones controladas entre validación, generación, ejecución e interpretación. Esta estructura resulta especialmente útil cuando se combinan componentes generativos con comprobaciones programáticas, porque permite distinguir con más claridad qué parte del proceso ha fallado y qué evidencias quedan disponibles para revisarlo.

El sistema desarrollado adopta ese patrón, pero de forma deliberadamente acotada. El modelo no selecciona libremente qué herramienta usar ni altera el orden global del proceso. En su lugar, opera dentro de un recorrido fijo con estado compartido y transiciones explícitas entre resolución de datos, planificación, generación de código, validación previa a la ejecución, ejecución controlada e interpretación final. Esta decisión reduce la libertad operativa, pero refuerza el control y la trazabilidad, que son dos prioridades centrales del trabajo.

2.4. Generación y ejecución controlada de código

La generación automática de código es una de las capacidades que más ha impulsado el uso de los modelos de lenguaje en tareas técnicas. Uno de los trabajos de referencia en esta línea fue Codex, que ayudó a consolidar la idea de que un modelo de lenguaje puede recibir una instrucción escrita en lenguaje natural, transformarla en un programa y comprobar después si ese programa funciona mediante pruebas funcionales [3]. En ciencia de datos, DS-1000 amplió esta línea con un banco de pruebas más cercano a escenarios reales y apoyado en librerías habituales de análisis [7].

Una idea especialmente relevante para este proyecto es la de PAL, donde el modelo no resuelve todo el problema en lenguaje natural, sino que transforma la petición en un programa ejecutable y delega el cálculo efectivo en un intérprete [5]. Esta perspectiva encaja de forma directa con la arquitectura desarrollada: aquí el código no se trata como una respuesta final, sino como un artefacto intermedio que debe ser validado, ejecutado y contrastado antes de redactar la respuesta al usuario.

La literatura reciente también ha mostrado que la reparación iterativa puede mejorar los resultados cuando el primer intento falla. Self-Debugging explora cómo un modelo puede revisar su propio programa a partir de su explicación y de la evidencia de ejecución [4]. Sin reproducir exactamente ese planteamiento, el sistema retoma la misma intuición: cuando aparecen errores reparables, conviene devolver al modelo un contexto acotado del fallo y exigir una nueva versión del script bajo restricciones explícitas.

Ejecutar código generado por un modelo introduce, además, riesgos específicos. Los estudios sobre asistentes de programación han mostrado que sus sugerencias pueden contener vulnerabilidades, incluso cuando parecen funcionales [12]. Por este motivo, el código no se acepta por mera plausibilidad. La propuesta lo obliga a respetar un contrato estructurado, lo somete a una validación previa y limita el espacio de operaciones permitidas antes de su ejecución efectiva.

2.5. Fiabilidad y evaluación de sistemas generativos

Una limitación conocida de los modelos generativos es la producción de respuestas plausibles que no se corresponden con la información disponible, fenómeno denominado habitualmente alucinación [6]. La generación aumentada con recuperación busca reducir este problema apoyando la respuesta en fuentes obtenidas de forma explícita [8]. Aunque el sistema no incorpora recuperación documental, comparte el mismo principio de anclaje: la explicación final debe apoyarse en datos locales, código ejecutado y resultados registrados.

Este anclaje es especialmente importante en el análisis financiero. Cuando una res-

puesta presenta datos de rentabilidad, volatilidad o *drawdown*, debe ser posible revisar tanto la información de partida como el procedimiento seguido. También conviene separar con claridad la descripción del comportamiento histórico de cualquier predicción o recomendación. Conservar la consulta, el plan, el código, los resultados, los errores y la respuesta final no elimina la posibilidad de fallo, pero ayuda a localizarlo y a valorar sus consecuencias.

La evaluación de un sistema de estas características no puede reducirse a una única cifra. HELM defiende marcos de evaluación amplios y transparentes, apoyados en varios escenarios y métricas, para evitar que un resultado agregado oculte limitaciones relevantes [9]. De forma complementaria, la revisión de evaluación útil de LLM distingue entre valorar la capacidad nuclear del modelo y valorar su comportamiento como agente integrado en un sistema [13]. Esta distinción resulta muy pertinente en este trabajo, porque aquí importan tanto la calidad de la salida final como la robustez del flujo, la pertinencia del plan, la corrección del código y la trazabilidad de cada ejecución.

En el caso del texto generado, también se ha señalado que las métricas automáticas tradicionales no siempre reflejan bien la calidad percibida durante una revisión humana [10]. Siguiendo estas ideas, la evaluación del proyecto combina indicadores técnicos, revisión cualitativa y lectura humana de resultados. Esta aproximación permite distinguir entre los fallos del flujo y las respuestas que, aun completándose, presentan problemas de pertinencia, fidelidad o claridad.

2.6. Comparativa y posicionamiento de la propuesta

La Tabla 2.1 resume las líneas revisadas y muestra cómo se relacionan con el sistema desarrollado. El objetivo concreto es integrar un modelo de lenguaje con generación controlada de código y análisis reproducible de datos históricos.

Tabla 2.1: Comparación entre los trabajos relacionados y la propuesta desarrollada.

Trabajo o línea	Aportación principal	Relación con este trabajo
FinBERT, BloombergGPT y FinGPT [1, 16, 19]	Modelos entrenados o adaptados al lenguaje financiero.	Se centran en el modelo; la propuesta estudia la integración controlada de un modelo abierto generalista dentro de un flujo verificable.
FinRobot y TradingAgents [17, 21]	Agentes para investigación, valoración o <i>trading</i> .	Inspiran la separación de responsabilidades, pero el sistema no toma decisiones de inversión ni actúa sobre el mercado.
ReAct, Toolformer y revisiones de agentes [14, 15, 20]	Uso de herramientas externas y coordinación de pasos mediante observaciones y transiciones controladas.	Respaldan el uso de herramientas y un estado común; aquí el recorrido está fijado para favorecer el control.

Trabajo o línea	Aportación principal	Relación con este trabajo
Codex, DS-1000, PAL y Self-Debugging [3–5, 7]	Síntesis de código, evaluación en escenarios técnicos y reparación iterativa de programas.	Justifican tratar el código como artefacto intermedio, validarlo antes de ejecutarlo y permitir reparaciones acotadas cuando aparecen errores.
Seguridad de código generado [12]	Riesgos de vulnerabilidades o implementaciones inseguras en sugerencias producidas por modelos.	Refuerza la necesidad de restringir operaciones, validar el script y no ejecutar código por mera plausibilidad.
Alucinaciones, RAG y evaluación de LLM [6, 8–10, 13]	Estudio de respuestas no fieles, mecanismos de anclaje y evaluación multidimensional del comportamiento del sistema.	Fundamentan la combinación de anclaje a datos locales, indicadores técnicos, dimensiones cualitativas y revisión humana.

Capítulo 3

Material de trabajo y planteamiento experimental

Este capítulo sitúa el punto de partida práctico del trabajo antes de entrar en la metodología formal y antes de describir con detalle el flujo multiagente. Su función es precisar con qué material se construye el sistema, qué clase de consultas se quiere estudiar y por qué ese conjunto resulta suficiente para poner a prueba la arquitectura en condiciones realistas.

3.1. Origen y naturaleza del material de trabajo

La fuente principal de datos históricos utilizada en el proyecto es Yahoo Finance, consumida a través de la librería `yfinance` [2, 18]. Esta elección permite trabajar con una base amplia de instrumentos y facilita que el sistema resuelva consultas abiertas sin depender de un pequeño catálogo manual preparado de antemano. La arquitectura se plantea como un flujo capaz de interpretar la consulta del usuario, convertirla en una petición estructurada y comprobar después si los datos históricos necesarios pueden descargarse de forma operativa.

Esto no significa que el sistema pueda analizar cualquier cosa sin restricciones. El alcance real depende de que el instrumento solicitado pueda identificarse con suficiente claridad, de que exista histórico accesible en `yfinance` y de que la petición sea compatible con un análisis basado en precios y volumen. Dicho de otro modo, el trabajo no fija una lista cerrada de activos, pero sí se mueve dentro del universo de consultas que pueden resolverse de manera razonable con datos históricos de mercado.

Para estudiar ese comportamiento se ha buscado una combinación de apertura y representatividad. Por un lado, la entrada del sistema sigue siendo una consulta libre en lenguaje natural. Por otro, el proyecto se apoya en familias de casos que permiten tensionar la arquitectura de forma controlada. En la práctica, esto incluye

ejemplos sobre acciones, fondos cotizados, índices, divisas, una materia prima y un criptoactivo, con distintos horizontes temporales y con granularidad diaria e intradía. Esta variedad no pretende cubrir todo el mercado financiero, pero sí ofrecer un conjunto lo bastante heterogéneo como para observar cómo responde el sistema ante consultas simples, comparativas y más exigentes.

Otro rasgo importante es que los datos no se consideran solo una entrada pasiva, sino una parte activa del experimento. La fase de datos debe resolver el instrumento, validar la petición y completar una descarga real antes de que comience la parte analítica. Por eso el material de trabajo no se reduce a la consulta del usuario ni a la respuesta final: incluye también los artefactos generados durante la descarga, la normalización, la ejecución del código y la traza completa del proceso.

3.2. Casos y escenario experimental de partida

Durante la construcción del sistema se utilizaron varios ejemplos representativos para comprobar si cada fase iba funcionando como se esperaba. Estos ejemplos no constituyen la evaluación final, pero sí ayudan a entender el tipo de material empleado para poner a prueba el proyecto. Entre ellos aparecen consultas de crecimiento simple sobre un único activo, comparaciones entre dos instrumentos, lecturas generales de corto plazo, análisis de retorno y riesgo, y algunos casos con mayor carga interpretativa o con restricciones de formato más explícitas. En este sentido, los ejemplos de desarrollo sirvieron como banco de pruebas incremental para verificar que la fase de datos, la planificación analítica, la generación de código, la validación y la interpretación final mantenían una coherencia razonable.

La evaluación principal se articula, sin embargo, alrededor de una batería de 50 consultas organizada en tres niveles de complejidad. El nivel A agrupa consultas directas, centradas en una o dos métricas fáciles de identificar y con un formato de respuesta simple. El nivel B introduce comparaciones más ricas, varias métricas relevantes o una estructura de salida algo más elaborada. El nivel C reúne los casos más exigentes: informes más completos, restricciones explícitas de presentación o peticiones donde el sistema debe mantener mejor la disciplina de salida y la prudencia interpretativa.

La distribución prevista de la batería no es equilibrada entre niveles, sino intencionadamente más exigente: 10 casos de nivel A, 20 de nivel B y 20 de nivel C. La razón es sencilla. Aunque parece razonable esperar mejor rendimiento en las consultas más fáciles, interesa comprobar qué ocurre cuando aumenta la dificultad y la respuesta exige una mejor coordinación entre la fase de datos, la parte analítica y la interpretación final. La batería no se diseña solo para confirmar que el sistema responde en casos sencillos, sino para identificar en qué punto empieza a degradarse su comportamiento y qué errores emergen cuando el escenario se vuelve más complejo.

Aunque la entrada del sistema siga siendo abierta, las consultas que se quieren estudiar pueden agruparse en varias familias. La primera corresponde a consultas descriptivas directas, como crecimiento acumulado, rango de precios o resumen general de un activo en un periodo dado. La segunda incluye comparativas entre instrumentos, por ejemplo retorno frente a riesgo, *drawdown*, correlación o liderazgo relativo. La tercera se apoya en lecturas algo más técnicas, como medias móviles, perfil de volatilidad o comportamiento por subperiodos. La cuarta incorpora restricciones explícitas de formato o de estilo, por ejemplo separar métricas y limitaciones, redactar para un usuario no técnico o devolver estructuras concretas de salida. Por último, algunos casos fuerzan al sistema a reconocer mejor los límites del propio análisis histórico, evitando noticias, predicciones o recomendaciones de inversión.

En conjunto, este planteamiento experimental resulta suficiente para justificar el estudio por tres razones. En primer lugar, el material utilizado refleja el comportamiento real del sistema, que depende de consultas abiertas y de descargas efectivas de datos históricos. En segundo lugar, la variedad de activos y de tareas permite observar el flujo en situaciones distintas sin convertir el trabajo en una exploración inabarcable. En tercer lugar, la batería escalonada por niveles de dificultad no solo mide si el sistema acierta, sino también cómo cambia su rendimiento cuando la consulta exige más cálculo, más estructura y una interpretación más cuidadosa.

Capítulo 4

Metodología

Este capítulo define cómo se estudia el sistema y con qué criterio se juzgan sus resultados. No describe el funcionamiento interno del flujo, que se desarrolla en el capítulo siguiente, sino el marco con el que se analiza su comportamiento de una forma defendible. La idea central es valorar la viabilidad de una arquitectura basada en LLM que prioriza la trazabilidad y la explicabilidad frente a una respuesta generativa directa y opaca.

4.1. Enfoque metodológico

El trabajo se plantea como una evaluación empírica de un sistema orientado al análisis financiero histórico a partir de consultas en lenguaje natural. El interés principal no es medir la máxima capacidad posible de un modelo aislado, sino estudiar si una arquitectura más controlada, compuesta por fases separadas y artefactos observables, puede producir respuestas útiles, comprensibles y prudentes para el usuario final.

Desde este punto de vista, la metodología no se centra solo en si el sistema “responde”, sino en dos cuestiones complementarias. La primera es si logra completar una ejecución de extremo a extremo y entregar una salida final utilizable. La segunda es si esa respuesta final mantiene una calidad suficiente en relación con lo que pedía la consulta. Esta doble lectura obliga a distinguir entre completar técnicamente el flujo y obtener un resultado satisfactorio desde la perspectiva del usuario.

4.2. Pregunta de investigación y objetivo evaluable

La pregunta que orienta este capítulo puede formularse del siguiente modo: *evaluar la viabilidad de una arquitectura LLM trazable y explicable para resolver consultas de análisis financiero histórico en lenguaje natural.*

Esta formulación es coherente con el objetivo general del trabajo, que no busca

maximizar la capacidad bruta de un modelo poco interpretable, sino comprobar si es posible construir un sistema más controlado, capaz de dejar evidencias del proceso seguido y de acotar mejor dónde y por qué se producen los errores.

En consecuencia, el objetivo evaluable no consiste en demostrar que el sistema puede responder a cualquier consulta financiera, ni en compararlo con un analista experto, ni en presentarlo como herramienta de predicción o asesoramiento. Lo que se quiere comprobar es si el flujo propuesto resulta viable como sistema de apoyo para análisis histórico: esto es, si puede resolver un conjunto representativo de consultas, mantener un comportamiento trazable y producir respuestas finales suficientemente adecuadas para el uso planteado.

4.3. Unidad de análisis

La unidad de análisis principal es una consulta completa ejecutada de extremo a extremo. Esta decisión implica que el caso evaluado no es una etapa aislada del workflow ni un artefacto intermedio concreto, sino el recorrido completo que va desde la petición inicial hasta la respuesta final.

Esta elección responde a una razón simple. El sistema está pensado para que el usuario formule una consulta y reciba una respuesta final. Por tanto, no tiene sentido considerar como caso satisfactorio una ejecución que no llega a producir una salida utilizable para el usuario, aunque algunas fases internas hayan funcionado correctamente. Del mismo modo, tampoco basta con llegar al final del flujo si la respuesta obtenida no mantiene una calidad suficiente en relación con la consulta planteada.

4.4. Diseño experimental

La evaluación se apoya en la batería de 50 consultas descrita en el capítulo anterior. Esa batería se organiza en tres niveles de complejidad, A, B y C, y se distribuye de forma deliberadamente exigente en 10 casos de nivel A, 20 de nivel B y 20 de nivel C. Esta estructura permite observar no solo el comportamiento del sistema en consultas relativamente directas, sino también su respuesta cuando aumentan las exigencias analíticas, de formato o de prudencia interpretativa.

Cada caso se ejecuta una única vez y todos los ejemplos se lanzan con la misma configuración del sistema y del modelo. En particular, la evaluación se realiza con el modelo `openai/gpt-oss-20b` [11], manteniendo constante la configuración general para toda la batería. De este modo, las diferencias observadas entre casos pueden interpretarse en función de la dificultad de la consulta y del comportamiento del flujo, y no como consecuencia de cambiar de modelo o de criterio de ejecución entre ejemplos.

4.5. Protocolo general de ejecución

El protocolo de evaluación sigue una misma lógica para todos los casos. A partir de una consulta en lenguaje natural, el sistema intenta completar el recorrido de extremo a extremo: resolución de datos, descarga, análisis, generación y validación del código, ejecución e interpretación final. Si el flujo consigue terminar y entregar una respuesta final utilizable, el caso pasa a la fase de evaluación cualitativa. Si no lo consigue, el caso se registra como no completado y se analiza aparte según la familia de error observada.

Aunque el tiempo de ejecución de cada caso se conserva como información complementaria, su interpretación principal se reserva para el capítulo de resultados.

4.6. Criterio general de éxito

Un caso no se considera exitoso solo por llegar a una respuesta final. Para que el resultado se considere satisfactorio dentro del trabajo deben cumplirse dos condiciones. La primera es que el flujo complete la ejecución y produzca una salida final utilizable. La segunda es que esa respuesta alcance una calidad suficiente en la revisión manual posterior.

En este sentido, se adopta como criterio de referencia que un resultado satisfactorio es aquel que obtiene al menos 8 puntos sobre 10 en la rúbrica de evaluación. Alcanzar ese umbral significa que la respuesta final presenta una calidad suficientemente alta como para considerarla adecuada, clara y prudente dentro del alcance del sistema. Las respuestas que completan el flujo pero no llegan a ese nivel no se consideran fallos técnicos del mismo tipo que los casos no completados, pero sí se interpretan como resultados que todavía requieren mejora para alcanzar una calidad suficiente.

4.7. Evaluación de casos no completados

Los casos que no consiguen producir una respuesta final utilizable no se evalúan con la rúbrica cualitativa. En su lugar, se analizan por separado según la familia de fallo observada. Esta clasificación no pretende fijar una taxonomía cerrada desde el principio, sino ofrecer un marco inicial ordenado para estudiar dónde se concentran los bloqueos del sistema. Como familias generales de partida se consideran las siguientes:

- fallo en resolución o descarga de datos;
- fallo en generación o validación del análisis;
- fallo de ejecución;
- fallo en la obtención de una salida final utilizable.

La razón para trabajar con familias y no solo con errores aislados es que dos casos distintos pueden fallar por mensajes concretos diferentes y, sin embargo, pertenecer al mismo tipo de problema metodológicamente relevante. Esta agrupación permite analizar tendencias generales sin perder de vista que, al revisar los resultados reales de la batería, podrán aparecer matices o subtipos adicionales. Por tanto, las familias de error se fijan aquí como un marco inicial abierto a refinarse si los resultados muestran situaciones nuevas o diferencias que merezcan una separación más precisa.

4.8. Evaluación de casos completados

Solo los casos que llegan a una respuesta final pasan a la evaluación cualitativa. En ellos, el interés principal no es inspeccionar artefactos internos que el usuario no ve, sino juzgar la calidad de la salida final que el sistema entrega.

La rúbrica se apoya en cinco criterios, todos ellos ponderados por igual:

- adecuación a la consulta;
- cobertura analítica;
- coherencia y solidez aparente;
- claridad y utilidad comunicativa;
- prudencia y tratamiento de limitaciones.

El primer criterio valora si la respuesta responde realmente a lo que pedía el usuario, incluyendo activos, periodo, comparación, tono o formato cuando esos elementos se exigen de manera explícita. El segundo comprueba si la respuesta cubre los componentes analíticos esenciales de la consulta, sin premiar la longitud por sí misma. El tercero se centra en la coherencia interna del resultado y en si las cifras, relaciones y conclusiones aparecen integradas de forma consistente. El cuarto estudia si la salida es comprensible y útil para un lector no especializado. El quinto revisa si la respuesta mantiene la prudencia exigible a un sistema de análisis histórico y si trata correctamente sus limitaciones.

Para que esta evaluación resulte interpretable, cada criterio debe entenderse de forma operativa. No se trata de asignar una impresión general difusa, sino de comprobar qué aparece realmente en la respuesta final y cómo se ajusta a la consulta planteada.

- **Adecuación a la consulta.** Evalúa si la respuesta contesta a lo que el usuario pidió. Aquí se revisa si aparecen los activos correctos, el periodo o comparación solicitados y, cuando procede, el formato pedido por el usuario. Se asigna un 0 cuando la respuesta falla en elementos esenciales de la consulta; un 1 cuando

responde solo de forma parcial o altera alguna parte importante; y un 2 cuando responde de manera clara y ajustada a los requisitos principales del caso.

- **Cobertura analítica.** Evalúa si la respuesta incluye los elementos analíticos que hacían falta para contestar bien a la consulta. No premia decir muchas cosas, sino cubrir lo necesario. Se asigna un 0 cuando faltan componentes esenciales del análisis; un 1 cuando aparece una parte relevante pero se omite algún elemento importante; y un 2 cuando la respuesta recoge de forma suficiente las métricas, comparaciones o bloques que el caso exigía.
- **Coherencia y solidez aparente.** Evalúa si la respuesta mantiene coherencia interna y si sus cifras, relaciones y conclusiones parecen bien integradas. No equivale a una auditoría financiera experta, pero sí exige consistencia visible. Se asigna un 0 cuando hay contradicciones claras, cifras mal integradas o conclusiones incoherentes; un 1 cuando la respuesta es en general razonable, pero presenta alguna imprecisión o una relación débil entre datos y conclusión; y un 2 cuando el resultado aparece consistente y bien construido desde el punto de vista del lector.
- **Claridad y utilidad comunicativa.** Evalúa si la respuesta puede entenderse con facilidad y si resulta útil para un lector no especializado. Se asigna un 0 cuando la respuesta es confusa, desordenada o poco aprovechable; un 1 cuando se entiende, pero con carencias de orden o explicación; y un 2 cuando la salida es clara, legible y transmite una idea útil para interpretar el resultado.
- **Prudencia y tratamiento de limitaciones.** Evalúa si la respuesta respeta el alcance del sistema, evita recomendaciones o predicciones impropias y trata correctamente las limitaciones del análisis histórico. Se asigna un 0 cuando incurre en afirmaciones que exceden ese alcance; un 1 cuando mantiene cierta prudencia, pero trata las limitaciones de forma insuficiente; y un 2 cuando conserva un tono prudente, reconoce los límites relevantes y no presenta el resultado como asesoramiento financiero.

4.9. Escala de puntuación y rúbrica

Cada uno de los cinco criterios anteriores se puntúa con una escala discreta de 0, 1 o 2. Esta elección se adopta por razones de interpretabilidad y consistencia. Una escala corta facilita distinguir entre incumplimiento claro, cumplimiento parcial y cumplimiento adecuado, y evita una falsa precisión que sería difícil de sostener en una evaluación humana de este tipo.

En esta escala, el valor 0 corresponde a un incumplimiento claro del criterio; el valor 1 indica un cumplimiento parcial o insuficiente; y el valor 2 representa un cumplimiento

adecuado. Esta gradación permite objetivar en la medida de lo posible una revisión que sigue siendo humana, ya que obliga a distinguir entre respuestas claramente incorrectas, respuestas aceptables pero todavía mejorables y respuestas que satisfacen bien el criterio evaluado.

La puntuación total máxima es de 10 puntos. Todos los criterios pesan lo mismo, ya que el objetivo no es imponer una ponderación artificial entre dimensiones, sino obtener una lectura equilibrada de la calidad final de la respuesta. En este marco, los resultados se interpretan de la siguiente forma:

- de 8 a 10 puntos: resultado satisfactorio;
- de 6 a 7 puntos: caso completado con calidad insuficiente;
- de 0 a 5 puntos: caso completado con calidad baja.

De esta manera, la metodología separa con claridad tres situaciones distintas: casos no completados, casos completados pero todavía insuficientes y casos completados con una calidad final satisfactoria.

4.10. Revisión humana

La aplicación de la rúbrica se realiza mediante revisión humana llevada a cabo por el autor del trabajo. Esta decisión se justifica porque varios de los criterios evaluados —como la adecuación a la consulta, la claridad de la respuesta o la prudencia interpretativa— requieren una lectura contextual que no conviene reducir a una única comprobación automática.

Al mismo tiempo, esta revisión no debe interpretarse como una auditoría financiera experta ni como una certificación profesional del contenido económico. Su función es valorar el comportamiento del sistema dentro del marco del trabajo y determinar si la respuesta final resulta suficientemente adecuada para el objetivo planteado. Precisamente por ello, la rúbrica se formula con criterios explícitos y una escala simple, buscando que el juicio manual sea lo más interpretable posible.

4.11. Amenazas a la validez

La primera limitación metodológica es que la revisión cualitativa depende del juicio del autor del trabajo. Aunque la rúbrica reduce parte de esa subjetividad al definir criterios explícitos y una escala cerrada, el resultado sigue descansando en una evaluación humana no experta en finanzas. Por tanto, las puntuaciones deben leerse como una valoración aplicada del comportamiento del sistema y no como una auditoría financiera externa.

La segunda limitación es la dependencia de una única configuración del sistema y de un único modelo, `openai/gpt-oss-20b` [11]. Esta decisión favorece la comparabilidad interna de la batería, pero restringe el alcance de las conclusiones: los resultados obtenidos no deben generalizarse automáticamente a otras configuraciones, otros proveedores o modelos con distinto perfil de capacidad.

La tercera limitación procede del propio material de evaluación. Aunque la batería de 50 consultas cubre diferentes niveles de dificultad y varios tipos de instrumento, sigue siendo un conjunto finito y deliberadamente acotado. En consecuencia, la metodología permite estudiar la viabilidad del sistema en ese marco concreto, pero no demostrar un comportamiento universal frente a cualquier consulta financiera posible.

Por último, el alcance metodológico de la evaluación se limita al análisis histórico. Los resultados no deben extrapolarse a tareas de predicción, asesoramiento financiero o interpretación de factores externos. Las implicaciones generales de este límite se desarrollan con mayor amplitud en el capítulo de limitaciones.

Capítulo 5

Flujo multiagente del sistema

En este capítulo se describe el funcionamiento interno del sistema multiagente propuesto. El objetivo es explicar cómo una consulta en lenguaje natural se transforma progresivamente en una respuesta final apoyada en datos históricos, artefactos intermedios y mecanismos de validación. Para ello, primero se presenta una visión general del flujo completo y, a continuación, se detalla el recorrido de una ejecución de extremo a extremo.

5.1. Visión general del sistema

La Figura 5.1 resume la arquitectura general del sistema. Esa figura se corresponde con la implementación efectiva del `SimpleFinancialWorkflow`, cuyo pipeline recorre nueve nodos principales en este orden:

1. validación inicial de la consulta (`ingest_node`);
2. planificación de la petición de datos (Agente 1);
3. validación estructural del `FinancialDataRequest`;
4. descarga real y validación operativa de datos;
5. planificación analítica (Agente 2);
6. generación de código (Agente 3);
7. validación del código (Agente 4);
8. ejecución controlada y validación de runtime;
9. interpretación final (Agente 5).

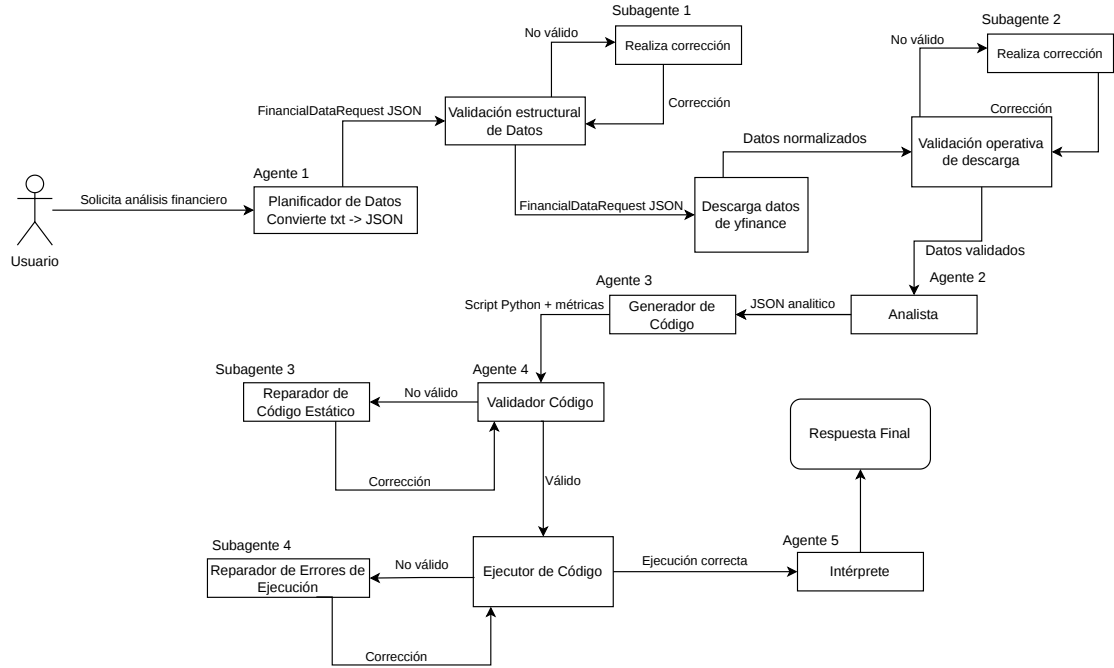


Figura 5.1: Flujo multiagente de referencia para el desarrollo del sistema de análisis financiero.

La lógica representada en la figura puede entenderse en cuatro bloques principales. El primero corresponde a la resolución del problema de datos. A partir de la consulta original, el sistema comprueba que la entrada sea utilizable, transforma esa petición en un **FinancialDataRequest**, verifica su coherencia estructural y, finalmente, intenta la descarga real de los datos históricos necesarios. En esta parte, el objetivo no es todavía producir una interpretación financiera, sino asegurar que el resto del flujo trabajará sobre una base de datos efectivamente descargada y validada.

El segundo bloque abarca la planificación analítica y la generación de código. Una vez que la fase de datos ha dejado un contexto operativo estable, el Agente 2 define qué debe calcularse y con qué requisitos de salida, mientras que el Agente 3 traduce ese plan a un script Python ejecutable. Esta separación entre planificación e implementación es una de las decisiones centrales de la arquitectura, porque permite distinguir entre el razonamiento analítico y la materialización técnica de ese razonamiento.

El tercer bloque está dedicado al control del código generado. Antes de que el script se ejecute, el Agente 4 comprueba si el resultado es aceptable, si requiere reparación o si debe bloquearse. Superada esa puerta, el sistema lanza la ejecución real y valida la evidencia observada en runtime, tanto en términos de terminación correcta del proceso como de adecuación de la salida estructurada obtenida. De este modo, la arquitectura no se limita a comprobar si el código parece razonable, sino que exige también que produzca una salida reutilizable por la fase final.

El cuarto y último bloque corresponde a la interpretación. Cuando la ejecución ya ha dejado una salida considerada válida, el Agente 5 la transforma en una respuesta final legible para el usuario. Esta fase no recalcula métricas ni reabre el problema analítico, sino que redacta la respuesta a partir de los resultados efectivamente obtenidos y del contexto resuelto durante el flujo.

La figura también muestra que el sistema no se apoya únicamente en una ruta ideal. Existen mecanismos de corrección asociados a distintas familias de fallo: problemas estructurales en la petición de datos, problemas operativos de descarga, defectos detectados en la validación del código y errores observados durante la ejecución real. Esta organización permite que la reparación se produzca sobre el artefacto afectado, sin rehacer innecesariamente todo el recorrido.

De forma resumida, los subagentes solo entran en acción cuando una fase deja un artefacto reparable, pero todavía no bloqueado. Si el `FinancialDataRequest` falla por razones estructurales, actúa el Subagente 1 y puede corregirlo hasta dos veces. Si el request es formalmente válido, pero la descarga real no produce datos utilizables, interviene el Subagente 2, también con un máximo de dos reparaciones. Cuando el problema está en el script generado antes de ejecutarse, entra el Subagente 3 y se permiten hasta dos correcciones de código. Por último, si el fallo aparece ya en runtime, el Subagente 4 repara el script a partir de la evidencia real de ejecución, dentro de un límite de tres intentos totales de ejecución.

En conjunto, la visión global que ofrece la Figura 5.1 permite entender el rasgo principal del sistema: la respuesta final no surge de una llamada monolítica al modelo, sino de una cadena de fases con contratos diferenciados, validaciones explícitas y artefactos observables. A partir de esta visión de conjunto, el siguiente apartado describe el recorrido completo de una ejecución real dentro de la arquitectura.

5.2. Recorrido completo de una ejecución

Para que este capítulo no se quede en una descripción teórica del flujo, en este apartado se sigue una ejecución real de principio a fin. La consulta elegida fue: *“Analiza AAPL en 3 meses y explica qué se puede concluir solo con estos datos históricos. No uses noticias, no predigas el futuro y no recomiendes comprar ni vender”*.

5.2.1. Fase 1. Resolución y validación de datos

La primera fase arranca con una entrada mínima. En este punto el sistema todavía no conoce el ticker final, ni el periodo, ni la ruta del CSV que terminará usando más adelante. Lo primero que hace es construir el estado inicial de la ejecución:

{

```

"user_query": "Analiza AAPL en 3 meses y explica que se puede concluir solo con estos datos
  historicos. No uses noticias, no predigas el futuro y no recomiendes comprar ni vender",
"normalized_query": {
  "query": "Analiza AAPL en 3 meses y explica que se puede concluir solo con estos datos historicos.
    No uses noticias, no predigas el futuro y no recomiendes comprar ni vender",
  "tickers": [],
  "start": null,
  "end": null,
  "period": null,
  "interval": null,
  "needs_clarification": false,
  "csv_paths": [],
  "warnings": []
},
"csv_paths": [],
"status": "created"
}

```

Aquí se ve bien el papel de `WorkflowState`: al principio conserva la consulta original y poco más. A partir de ese punto, la fase de datos hace dos comprobaciones sucesivas. Primero se valida que la consulta exista y no llegue vacía. Después, cuando el Agente 1 propone una petición de datos, se comprueba localmente que esa petición tenga sentido antes de intentar ninguna descarga.

La salida del Agente 1, en este ejemplo, queda así:

```

{
  "user_query": "Analiza AAPL en 3 meses y explica que se puede concluir solo con estos datos
    historicos. No uses noticias, no predigas el futuro y no recomiendes comprar ni vender",
  "provider": "yfinance",
  "instruments": [
    {
      "ticker": "AAPL"
    }
  ],
  "interval": "1d",
  "start": null,
  "end": null,
  "period": "3mo",
  "required_fields": [
    "Open",
    "High",
    "Low",
    "Close",
    "Adj Close",
    "Volume"
  ],
  "needs_clarification": false,
  "clarification_reason": null
}

```

En esta parte todavía no se está comprobando si la descarga real funciona. Lo que se valida es que el request mantenga la consulta, use un proveedor soportado, incluya al menos un ticker, fije un intervalo permitido, no mezcle `period` con `start/end` y no quede bloqueado por una necesidad de aclaración. Solo cuando todo eso encaja se pasa a la descarga.

Después llega la normalización. Este paso es importante porque la salida del proveedor no se deja tal cual, sino que se transforma a una tabla canónica con columnas explícitas como Date, Ticker, Open, High, Low, Close, Adj Close y Volume. La razón es sencilla: el resto del pipeline necesita trabajar sobre un formato estable, sin depender de detalles propios del proveedor o de cómo vengan organizados los datos en origen.

Una vez descargados y normalizados, se realiza una validación operativa. En esta comprobación el sistema se fija, sobre todo, en cuatro cosas: que el dataset no esté vacío, que aparezcan las columnas esenciales, que el ticker pedido esté realmente presente y que la serie de precios sea utilizable.

Cuando esa fase termina bien, lo que le llega al Agente 2 no es el estado interno completo, sino un handoff explícito como este:

```
{
  "query": "Analiza AAPL en 3 meses y explica que se puede concluir solo con estos datos historicos. No
    uses noticias, no predigas el futuro y no recomiendes comprar ni vender",
  "tickers": [
    "AAPL"
  ],
  "temporal_context": {
    "start": null,
    "end": null,
    "period": "3mo",
    "interval": "1d"
  },
  "csv_paths": [
    "C:\\Users\\usuario\\Desktop\\tfm\\results\\data_normalized\\normalized_20260614_192501_769901.csv"
  ],
  "data_context": {
    "row_count": 64,
    "available_columns": [
      "Date",
      "Ticker",
      "Open",
      "High",
      "Low",
      "Close",
      "Adj Close",
      "Volume"
    ]
  },
  "warnings": [],
  "download_summary": {
    "provider": "yfinance",
    "tickers_requested": [
      "AAPL"
    ],
    "tickers_found": [
      "AAPL"
    ],
    "interval": "1d",
    "period": "3mo",
    "start": null,
    "end": null,
    "row_count": 64,
  }
}
```

```

    "columns": [
      "Date",
      "Ticker",
      "Open",
      "High",
      "Low",
      "Close",
      "Adj Close",
      "Volume"
    ]
  }
}

```

Este bloque resume bastante bien lo que deja la primera fase: la consulta ya está resuelta a un ticker concreto, el contexto temporal ya ha quedado fijado y el resto del sistema puede trabajar sobre una base de datos que ya ha sido descargada y validada.

5.2.2. Fase 2. Planificación analítica y generación de código

Con ese payload, el Agente 2 ya no razona sobre una consulta ambigua, sino sobre un contexto de datos que llega resuelto. Su salida en este ejemplo es la siguiente:

```

{
  "analytical_goal": "Analizar el comportamiento historico de AAPL durante los ultimos 3 meses y
    extraer conclusiones basadas unicamente en los datos descargados.",
  "analysis_type": "historical_overview",
  "metrics": [
    "average_close",
    "max_close",
    "min_close",
    "return_3mo",
    "volatility_3mo",
    "drawdown_3mo"
  ],
  "required_columns": [
    "Date",
    "Ticker",
    "Close",
    "Adj Close",
    "Volume"
  ],
  "output_requirements": [
    "metrics: JSON object containing keys average_close, max_close, min_close, return_3mo,
      volatility_3mo, drawdown_3mo.",
    "summary: plain text explanation of the main observations and conclusions that can be drawn from
      the data.",
    "limitations: list of limitations of the analysis."
  ],
  "reasoning": "La peticion solicita un analisis historico de AAPL en los ultimos 3 meses sin usar
    noticias ni hacer predicciones."
}

```

Lo importante aquí es que el Agente 2 no calcula todavía ninguna cifra. Su papel es fijar qué se quiere medir, con qué columnas y con qué forma de salida. Esa salida pasa después al Agente 3 junto con el mismo payload de la fase anterior.

El Agente 3 genera entonces el script Python que materializa ese plan. No hace falta copiar aquí el fichero completo, pero sí conviene enseñar un fragmento representativo:

```
payload = json.loads(Path(sys.argv[1]).read_text(encoding='utf-8'))
tickers = payload.get('tickers', [])
csv_paths = payload.get('csv_paths', [])

close = load_close_prices(csv_paths, tickers)
data = load_market_data(csv_paths, tickers)

avg_close = close_series.mean()
max_close = close_series.max()
min_close = close_series.min()
ret_3mo = (close_series.iloc[-1] - close_series.iloc[0]) / close_series.iloc[0]
vol_3mo = close_series.pct_change().std()

output = {
    "metrics": metrics,
    "summary": summary,
    "limitations": limitations
}
```

Este fragmento deja ver bien qué hace realmente el Agente 3. El script lee el payload desde disco, carga los datos con los helpers del proyecto, calcula las métricas que había fijado el Agente 2 y devuelve una salida estructurada. Dicho de otra forma, aquí ya no se está describiendo el análisis: se está convirtiendo en código ejecutable.

5.2.3. Fase 3. Validación y ejecución

La tercera fase empieza con la validación del código. En este ejemplo, el resultado que devuelve el Agente 4 es directamente válido:

```
{
  "decision": "valid",
  "errors": [],
  "warnings": [],
  "required_fixes": [],
  "reasoning": "El script implementa correctamente el plan de analisis y produce la salida esperada."
}
```

Por tanto, en este caso no fue necesario pasar por el Subagente 3. El script se aceptó a la primera y el workflow pudo continuar sin reparaciones en la parte de validación estática.

El ejecutor recibe entonces dos cosas: el script generado y el mismo payload operativo que ya se ha mostrado antes. A partir de ahí, la comprobación ya no es teórica. El sistema lanza el proceso de verdad y revisa después si la ejecución ha terminado bien, si `stdout` contiene un JSON parseable y si ese JSON conserva las claves obligatorias del workflow.

La salida obtenida en este ejemplo fue la siguiente:

```
{
```

```

"metrics": {
  "average_close": 278.00171875953674,
  "max_close": 315.20001220703125,
  "min_close": 246.6300048828125,
  "return_3mo": 0.16396134082127573,
  "volatility_3mo": 0.014529417415722213,
  "drawdown_3mo": -0.07820438914790316
},
"summary": "El analisis de los ultimos 3 meses de AAPL muestra que el precio de cierre promedio fue
278.00, con un maximo de 315.20 y un minimo de 246.63. El retorno total durante el periodo fue
16.40%. La volatilidad diaria se estima en 1.45%. El mayor drawdown observado fue -7.82%.",
"limitations": [
  "Los datos son de precios de cierre diarios sin considerar eventos corporativos, dividendos o
splits.",
  "No se incluyen factores externos ni noticias que puedan haber influido en el precio."
]
}

```

Aquí tampoco hizo falta activar al Subagente 4, porque la ejecución terminó bien en el primer intento y dejó una salida válida para la siguiente fase. Este punto es importante: la tercera fase no se limita a revisar si el código “parece razonable”, sino que comprueba si realmente funciona y si produce una salida reutilizable.

5.2.4. Fase 4. Interpretación final

La última fase toma la salida ejecutada y la prepara para el Agente 5. Lo que entra en el interpretador no es todo el estado interno, sino un payload más limpio, centrado en la consulta, el contexto resuelto y los resultados ya obtenidos:

```

{
  "user_query": "Analiza AAPL en 3 meses y explica que se puede concluir solo con estos datos
  historicos. No uses noticias, no predigas el futuro y no recomiendes comprar ni vender",
  "resolved_context": {
    "tickers": [
      "AAPL"
    ],
    "temporal_context": {
      "start": null,
      "end": null,
      "period": "3mo",
      "interval": "1d"
    }
  },
  "execution_output": {
    "metrics": {
      "average_close": 278.00171875953674,
      "max_close": 315.20001220703125,
      "min_close": 246.6300048828125,
      "return_3mo": 0.16396134082127573,
      "volatility_3mo": 0.014529417415722213,
      "drawdown_3mo": -0.07820438914790316
    },
    "summary": "El analisis de los ultimos 3 meses de AAPL muestra que el precio de cierre promedio fue
    278.00, con un maximo de 315.20 y un minimo de 246.63. El retorno total durante el periodo fue
    16.40%. La volatilidad diaria se estima en 1.45%. El mayor drawdown observado fue -7.82%.",
  }
}

```

```

    "limitations": [
        "Los datos son de precios de cierre diarios sin considerar eventos corporativos, dividendos o splits.",
        "No se incluyen factores externos ni noticias que puedan haber influido en el precio."
    ]
},
"warnings": []
}

```

Aquí se aprecia bien la idea de esta fase: el Agente 5 no vuelve a descargar datos, no rehace el análisis ni corrige el código. Lo que hace es tomar una salida ya ejecutada y convertirla en una respuesta final legible.

La respuesta final generada en esta traza queda así:

```

**Análisis de AAPL (últimos 3 meses)**

- **Precio de cierre promedio**: 278.00
- **Máximo**: 315.20
- **Mínimo**: 246.63
- **Retorno total**: 16.40 %
- **Volatilidad diaria**: 1.45 %
- **Drawdown máximo**: -7.82 %

**Conclusiones basadas únicamente en los datos históricos**
- El precio de AAPL ha mostrado una tendencia alcista, con un retorno positivo de más del 16 % en los últimos tres meses.
- La volatilidad diaria de alrededor del 1.45 % indica que los precios se movían moderadamente dentro de ese rango.
- El drawdown máximo de -7.82 % sugiere que la mayor caída desde un pico a un fondo fue relativamente contenida durante el periodo analizado.

**Limitaciones**
- Los datos corresponden únicamente a precios de cierre diarios y no incluyen eventos corporativos, dividendos ni splits.
- No se han considerado factores externos ni noticias que pudieran haber influido en el comportamiento del precio.

```

Con este último bloque se ve con bastante claridad el sentido del flujo completo. La respuesta final no aparece de forma inmediata tras la consulta del usuario, sino después de pasar por cuatro fases distintas: primero se resuelven y validan los datos, después se fija el plan analítico, luego se genera y ejecuta el código y, solo al final, se redacta la interpretación. Eso es precisamente lo que hace que el sistema sea más trazable y más defendible que una respuesta generada de una sola vez.

5.3. Mecanismos de corrección y criterios de parada

Una de las características más relevantes del sistema es que cada familia de fallo se gestiona con un mecanismo de corrección específico, en lugar de reintentar de forma indiferenciada toda la cadena. La Tabla 5.1 resume estas rutas.

Ruta	Se activa cuando	Artefacto corregido	Límite aplicado
Subagente 1	La validación estructural detecta un request reparable	request de datos	2 reparaciones estructurales
Subagente 2	La descarga real falla de forma operativamente recuperable	request de datos	2 reparaciones operativas
Subagente 3	El Agente 4 marca el código como reparable	script Python generado	2 reparaciones de código
Subagente 4	La ejecución real no deja una salida válida, pero el fallo parece recuperable	script Python generado	3 intentos totales de ejecución

Tabla 5.1: Rutas de reparación incorporadas en el workflow.

Lo primero que conviene fijar es que estas rutas no responden a una lógica binaria de acierto o error. En las validaciones de datos, en la validación de código y en la validación de ejecución aparece la misma semántica: **valid**, **reparable** y **blocked**. **valid** permite continuar; **reparable** abre una corrección automática; y **blocked** indica que, en esa ejecución concreta, el flujo deja de seguir corrigiendo y se detiene.

Esta semántica permite distinguir familias de fallo que, vistas desde fuera, podrían parecer similares. En la fase de datos, el Subagente 1 corrige forma y coherencia del **FinancialDataRequest**, mientras que el Subagente 2 actúa cuando el request ya era formalmente válido, pero la descarga real no ha producido una base utilizable. En la parte de código ocurre algo parecido: el Subagente 3 corrige scripts marcados como **reparable** por el Agente 4 antes de pasar a runtime, y el Subagente 4 corrige fallos observados ya durante la ejecución real.

La reparación, por tanto, es localizada. Cada subagente modifica solo el artefacto afectado y lo devuelve a su compuerta correspondiente: el request corregido vuelve a validarse o a descargarse, y el script corregido vuelve al Agente 4 o a una nueva ejecución. Esta decisión mejora el control y la trazabilidad, porque evita rehacer toda la cadena cuando el problema está acotado a una fase concreta.

Los límites de corrección también forman parte del diseño. En datos se permiten hasta dos reparaciones estructurales y dos operativas; en la parte de código, hasta dos reparaciones previas a la ejecución; y en runtime, hasta tres intentos totales de ejecución. Conviene distinguir estos límites de los reintentos internos usados para forzar

formato en una llamada LLM, que pueden llegar a tres dentro del mismo prompt. No es lo mismo que el modelo devuelva un JSON mal formado que descubrir, tras parsearlo bien, que la descarga no sirve o que el script falla al ejecutarse. Cada caso deja evidencias distintas y activa una respuesta distinta del workflow.

5.4. Prompts como contratos operativos

En este apartado se presentan los prompts que utilizan los agentes y subagentes del sistema. La idea es ver qué papel asume cada uno dentro del workflow, qué tipo de salida se le exige y qué límites se le imponen para que su función quede bien acotada.

5.4.1. Prompts de sistema compartidos

El primer nivel de contrato no depende del agente concreto, sino del bloque al que pertenece. La fase de datos comparte un prompt de sistema propio y el resto de la arquitectura comparte otro distinto, centrado ya en análisis histórico. Sus textos base son los siguientes:

Eres un componente de planificacion y correccion de peticiones de datos financieros historicos.
Tu objetivo es producir JSON valido, trazable y utilizable por codigo.
No calculas metricas financieras, no interpretas resultados y no redactas respuestas finales.

Eres un componente profesional de analisis financiero historico asistido por LLM.
Trabajas exclusivamente con datos historicos ya descargados y con el contrato JSON indicado.
No realizas predicciones, asesoramiento de inversion, recomendaciones de compra/venta,
analisis fundamental externo ni incorporas informacion que no este en la entrada.

Esta separación ya anticipa una idea importante del diseño: el sistema no trata igual la resolución de datos que la parte analítica. La primera se enfoca a producir requests trazables y descargables; la segunda se restringe a trabajar con datos ya resueltos.

5.4.2. Prompts de la fase de datos

En la fase de datos aparecen tres mensajes base: el del Agente 1 y los dos de reparación asociados a Subagente 1 y Subagente 2. En los tres casos, el contrato insiste en devolver solo JSON y en no convertir esta fase en un análisis financiero prematuro.

El texto base del Agente 1 es este:

Agente 1 - PLANIFICADOR DE DATOS.
Convierte la consulta del usuario en un FinancialDataRequest JSON valido.
Devuelve exclusivamente JSON, sin markdown ni texto adicional.
Tu trabajo es decidir que datos hay que pedir, no hacer el analisis financiero.
Debes:
1. Identificar instrumentos o tickers.
2. Inferir intervalo temporal y usar period o start/end segun corresponda.
3. Mantener provider='yfinance'.
4. Incluir required_fields con Open, High, Low, Close, Adj Close y Volume.

5. Marcar `needs_clarification=true` si la consulta no permite una descarga fiable.
No debes calcular metricas, razonar sobre rentabilidad ni proponer respuesta final.

Aquí el contrato es muy estricto: el agente puede decidir qué datos hacen falta, pero no puede empezar a analizar ni a contestar la consulta. Su única salida aceptable es un `FinancialDataRequest`.

El Subagente 1 reutiliza el mismo prompt de sistema, pero cambia el texto base para centrarse solo en la corrección estructural:

Subagente 1 - CORRECCION ESTRUCTURAL.
Devuelve exclusivamente un `FinancialDataRequest` JSON valido.
Corrige problemas de estructura, contrato, fechas, intervalos o instrumentos vacios.
Si no puedes corregir el problema de forma fiable, marca `needs_clarification=true`
y explica brevemente el motivo en `clarification_reason`.

Su alcance es deliberadamente estrecho. No intenta adivinar si la descarga real funcionará; solo corrige forma y coherencia del request.

El Subagente 2 comparte constructor con el anterior, pero cambia la directriz central para situarse en la validación operativa:

Subagente 2 - CORRECCION OPERATIVA.
Devuelve exclusivamente un `FinancialDataRequest` JSON valido.
Corrige el request para que la descarga real en yfinance sea util sin cambiar libremente la intencion del usuario.
Si no puedes corregir el problema de forma fiable, marca `needs_clarification=true`
y explica brevemente el motivo en `clarification_reason`.

La diferencia entre ambos subagentes queda muy clara si se leen en paralelo: uno corrige estructura y contrato; el otro corrige viabilidad real de descarga, pero sin reescribir arbitrariamente la intención del usuario.

5.4.3. Prompts de planificación, generación, validación y reparación

Una vez resuelta la parte de datos, el sistema pasa a la cadena analítica. Aquí no existe un único prompt genérico, sino varios contratos que separan con cuidado razonamiento analítico, implementación, validación y reparación.

El texto base del Agente 2 es el siguiente:

Agente 2 - ANALISTA.
Devuelve exclusivamente un objeto JSON valido con el esquema indicado. No incluyas markdown.
Tu tarea es convertir la peticion del usuario y los datos ya descargados en un plan verificable para un script Python posterior.
No calcules cifras y no redactes la respuesta final.
Debes apoyarte solo en el contexto de entrada ya resuelto por la fase de datos.
`required_columns` debe referirse a columnas de los datos disponibles.
`output_requirements` y `presentation_preferences` no pueden quedar vacios: deben reflejar de forma explicita
que salida estructurada devolvera el script y como debe presentarse el resultado.
En `output_requirements` debes describir siempre el contrato minimo del workflow:
`metrics` debe ser un objeto JSON, `summary` debe ser un texto plano y `limitations` debe ser una lista.

Si la consulta pide tabla, serie, ranking, drawdown u otros bloques estructurados, descríbelos como claves opcionales adicionales, pero sin sustituir metrics, summary ni limitations. No dejes output_requirements ni presentation_preferences en blanco ni con frases genéricas vacías.

Este prompt deja muy bien delimitado que el Agente 2 no calcula nada ni redacta la respuesta final. Su trabajo es dejar un AnalysisPlan lo bastante específico como para que el Agente 3 no tenga que reinterpretar la consulta. El Agente 3 recibe ese plan y lo convierte en un script. Su texto base es este:

Agente 3 - GENERADOR DE CODIGO.
Devuelve exclusivamente un objeto JSON válido con un único campo code. No incluyas markdown.
El valor code debe ser un script Python completo, autocontenido y ejecutable.
Debes implementar el AnalysisPlan recibido, no reinterpretar la consulta desde cero.
La prioridad es cumplir el contrato mínimo del workflow con un script robusto y simple.
No conviertas summary en un objeto, no uses metrics como lista y no sustituyas la salida mínima por tablas o series.

Aquí se aprecia bien la filosofía de esta parte del sistema: el código no se genera como una solución abierta, sino bajo un contrato muy explícito de entrada, carga de datos, imports permitidos y forma de salida.

Antes de ejecutar, el Agente 4 revisa el script generado con el siguiente texto base:

Agente 4 - VALIDADOR DE CODIGO.
Tu tarea es revisar el script generado y decidir si puede pasar a ejecución, si necesita corrección o si debe bloquearse.
Devuelve exclusivamente JSON válido con los campos decision, errors, warnings, required_fixes y reasoning. No incluyas markdown.
No ejecute el código. No reescribas el script. No cambies el AnalysisPlan.
Debes evaluar si el código implementa el plan recibido, si respeta el contexto de entrada y si mantiene una salida estructurada coherente con el workflow.
No confundas el esquema JSON de tu propia respuesta como validador con el JSON que debe imprimir el script.
El script debe imprimir metrics, summary y limitations, y opcionalmente analysis_type, tables, series o diagnostics.
Considera válidos, por contrato del workflow, los helpers importados desde src.execution.market_data cuando el script use solo load_close_prices, load_market_data, ticker_summary y make_json_safe.
No bloquee un script solo por mejoras opcionales si el contrato mínimo y la lógica principal están bien.

Esta redacción también es reveladora: el Agente 4 no propone código ni ejecuta nada, sino que separa explícitamente tres salidas posibles: **valid**, **repairable** o **blocked**. Además, el prompt insiste en no confundir el JSON que devuelve el validador con el JSON que debe imprimir el script, una distinción importante en la implementación actual.

Cuando el Agente 4 marca el caso como **repairable**, entra el Subagente 3 con este texto base:

Subagente 3 - REPARACION DE CODIGO.
El código anterior falló o fue rechazado. Devuelve exclusivamente JSON válido con un único campo code.
El script corregido debe ser completo, no un parche parcial.
No cambies el contrato de entrada ni el contrato de salida del workflow.
El script corregido debe imprimir un JSON con metrics, summary y limitations, y opcionalmente analysis_type, tables, series o diagnostics.

Asegura específicamente que `metrics` sea un objeto, `summary` sea un texto y `limitations` sea una lista. Si la consulta pedia tablas o series, mantenlas como claves opcionales, pero no sustituyas el contrato mínimo. Prefiere corregir con cambios pequeños y código simple antes que rehacer el script con más complejidad.

El Subagente 3 sigue trabajando con `analysis_plan` y con el feedback del validador, de modo que corrige el script antes de que llegue a runtime. El Subagente 4, en cambio, recibe un contrato distinto:

Subagente 4 - REPARADOR DE ERRORES DE EJECUCION.
Tu tarea es corregir un script Python que ya fue aceptado por la validación estática, pero que falló durante la ejecución real.
Devuelve exclusivamente JSON válido con un único campo `code`.
El script corregido debe ser completo, no un parche parcial.
No uses `analysis_plan` y no cambies el contrato de entrada ni el contrato de salida del workflow.
El script corregido debe imprimir un JSON con `metrics`, `summary` y `limitations`, y opcionalmente `analysis_type`, `tables`, `series` o `diagnostics`.
Asegura específicamente que `metrics` sea un objeto, `summary` sea un texto y `limitations` sea una lista.
Si el problema observado es de forma de salida, repara solo el contrato sin complicar innecesariamente la lógica analítica.
Si `stdout` ya estaba cerca de ser útil, prioriza conservar el contenido analítico y corregir la estructura.

La diferencia clave es que el Subagente 4 ya no recibe `analysis_plan`; trabaja con el error real de ejecución y con la evidencia de runtime. Así queda separada la reparación previa a la ejecución de la reparación posterior a un fallo observado.

5.4.4. Prompt del interpretador

El Agente 5 cierra el flujo con un contrato también deliberadamente acotado. Su texto base es este:

Agente 5 - INTERPRETE.
Redacta la respuesta final para el usuario usando solo la consulta original, el contexto resuelto mínimo, `execution_output` y los warnings relevantes.
No recalculés, no completes cifras ausentes y no incorpores datos externos.
Debes inferir por ti mismo cuánta elaboración necesita la respuesta según la query y según la riqueza real de los resultados obtenidos.
Si la consulta es simple, responde de forma simple y evita convertir la salida en un informe recargado. Si la consulta pide comparación, desglose o estructura, adapta el formato en consecuencia.
Si `execution_output` incluye `limitations` vacías, no fuerces una sección artificial de limitaciones. Si existen limitaciones relevantes, intégralas con naturalidad.

Este último prompt resulta especialmente interesante porque muestra una decisión de diseño importante del proyecto. A diferencia de los agentes anteriores, el interpretador no recibe `analysis_plan` ni otras pistas internas de la parte analítica. Solo recibe la consulta original, el contexto resuelto mínimo y la salida realmente ejecutada. Con ello, la interpretación final queda más anclada a los resultados observados y menos contaminada por metadatos internos del workflow.

Capítulo 6

Resultados y evaluacion

6.1. Funcion de este capitulo

Este capitulo debe mostrar que ha ocurrido al ejecutar el sistema y como deben interpretarse esos resultados. Su objetivo no es solo presentar metricas, sino explicar que ensena la evaluacion sobre la robustez del flujo y sobre el valor real del trabajo.

6.2. Enfoque recomendado

Conviene dejar este capitulo planteado, no cerrado. Cuando llegue el momento, la redaccion final deberia combinar:

1. una lectura global del rendimiento;
2. un analisis por tipo de caso o por grado de exigencia de la consulta;
3. una lectura por etapas del flujo;
4. una discusion de fallos, reparaciones y limites.

6.3. Preguntas que convendra responder mas adelante

1. ¿Que significa exactamente que el sistema haya completado bien un caso?
2. ¿Donde se concentran los errores: datos, planificacion, generacion, validacion, ejecucion o interpretacion?
3. ¿Que gana el sistema con los subagentes de correccion?
4. ¿Que tipos de consulta tensan mas la arquitectura?

5. ¿Que parte de los resultados respalda de verdad la tesis del trabajo?

6.4. Plantilla de estructura futura

6.4.1. Resultado global

Completar aqui el resumen global de la evaluacion cuando el sistema este implementado y probado.

6.4.2. Lectura por grupos de casos

Completar aqui la segmentacion de consultas que resulte mas util: por tipologia, por numero de restricciones, por formato de salida exigido o por etapa tensionada del flujo.

6.4.3. Analisis por etapas

Completar aqui el rendimiento observado en cada bloque del flujo.

6.4.4. Casos representativos

Completar aqui varios ejemplos que ilustren un caso sencillo, uno intermedio, uno complejo resuelto y un fallo informativo.

6.4.5. Discusion critica

Completar aqui que se ha demostrado, que no se ha demostrado y que mejoras quedarian priorizadas para una siguiente iteracion.

Capítulo 7

Limitaciones, aspectos éticos y reproducibilidad

Este capítulo recoge tres cuestiones que ayudan a situar correctamente el trabajo una vez vistos el flujo y la evaluación: qué límites tiene realmente el sistema, qué uso puede considerarse razonable dentro del alcance planteado y qué hace falta para repetir una ejecución con un grado suficiente de control. No se trata de añadir un cierre formal, sino de dejar claro hasta dónde llegan las conclusiones que pueden extraerse del proyecto y en qué condiciones conviene leerlas.

7.1. Limitaciones del estudio

La primera limitación es de alcance. El sistema se ha diseñado para resolver consultas de análisis histórico y descriptivo a partir de series de mercado. Por tanto, las conclusiones que puede ofrecer quedan acotadas a ese marco y no deben interpretarse como una valoración completa de un activo, sino como una lectura trazable de los datos que han entrado en una ejecución concreta.

La segunda limitación tiene que ver con la configuración experimental. La evaluación se ha realizado sobre una única arquitectura y una única configuración principal de modelo, lo que favorece la coherencia interna del estudio, pero reduce el alcance de la generalización. Aunque el flujo está separado por fases y dispone de subagentes de reparación, el comportamiento final sigue dependiendo en parte de la capacidad del modelo utilizado para interpretar la consulta, construir el plan analítico, generar código válido y redactar la interpretación final. Por ello, no puede asumirse sin más que los mismos resultados se mantendrían con otros modelos o proveedores.

La tercera limitación procede del propio material de evaluación. La batería de cincuenta consultas permite observar casos simples, comparativos y más exigentes, pero sigue siendo un conjunto acotado y construido para este trabajo. Sirve para estudiar

la viabilidad del sistema en escenarios representativos del proyecto, aunque no para demostrar un comportamiento universal frente a cualquier petición financiera posible.

También hay una limitación asociada a los datos. El flujo trabaja con descargas históricas obtenidas mediante `yfinance`, de modo que la disponibilidad, estabilidad y cobertura efectiva del proveedor condicionan el resultado. Además, las descargas relativas por periodo, como 3mo o 5y, dependen del momento en que se genera el CSV. Esto significa que, si no se fija un rango temporal cerrado o no se conserva el fichero ya descargado, una misma consulta puede apoyarse en ventanas de datos ligeramente distintas en ejecuciones realizadas en fechas diferentes.

Por último, conviene recordar que el sistema reduce errores, pero no los elimina por completo. El flujo valida el contrato de datos, comprueba la salida del análisis, somete el script a validación antes de ejecutarlo y conserva la evidencia de cada intento. Aun así, pueden aparecer fallos de interpretación, tratamientos insuficientes de las limitaciones del caso o dependencias no previstas del comportamiento del modelo. En consecuencia, el trabajo permite defender la utilidad de una arquitectura más controlada y trazable que una respuesta generativa directa, pero no extraer de ello una garantía plena de corrección en todos los escenarios.

7.2. Aspectos éticos y uso responsable

El sentido de este proyecto no es sustituir el juicio financiero humano, sino explorar si un flujo multiagente puede transformar una consulta libre en un análisis histórico trazable, con cálculo ejecutado y una salida final más controlada. Desde ese punto de vista, el uso responsable del sistema pasa por respetar el alcance con el que ha sido planteado y por no leer sus respuestas como si constituyeran asesoramiento financiero.

Esta restricción no queda solo en el plano declarativo, sino que aparece incorporada en el propio diseño del sistema. Los prompts base del análisis delimitan que el modelo debe trabajar exclusivamente con datos históricos ya descargados y, además, la evaluación incorpora un criterio de prudencia que penaliza respuestas que exceden ese marco.

Otro aspecto relevante es el riesgo asociado a ejecutar código generado por el modelo. En este trabajo el script no se ejecuta como una salida fiable por sí misma, sino dentro de un flujo que intenta acotar el problema: el código parte de un plan previo, se contrasta antes de ejecutarse, su salida debe respetar un contrato JSON mínimo y cada intento deja artefactos verificables. Estas medidas reducen el riesgo operativo y mejoran la inspección posterior, pero no convierten la ejecución en un proceso exento de riesgo. Por eso el sistema debe entenderse como una herramienta experimental y supervisada, adecuada para un entorno controlado y no como un mecanismo autónomo de toma de decisiones financieras.

En términos prácticos, la lectura responsable de una respuesta final pasa por no sobredimensionar lo que el sistema puede afirmar. Si una conclusión se apoya solo en precios históricos, entonces debe leerse exactamente en ese nivel: como una descripción de comportamiento pasado calculada de forma trazable, no como una garantía sobre lo que ocurrirá después ni como una recomendación de actuación.

7.3. Reproducibilidad

La reproducibilidad del trabajo descansa en dos planos complementarios. El primero es el entorno de ejecución: para repetir un caso hacen falta el mismo código del proyecto, una configuración equivalente del modelo, acceso a un endpoint compatible con OpenAI y disponibilidad de los datos históricos necesarios. Si alguno de estos elementos cambia, el comportamiento puede variar aunque la arquitectura general siga siendo la misma.

El segundo plano es la trazabilidad que deja cada ejecución. El workflow genera un `run_id` propio y conserva una carpeta de resultados con un `manifest`, un registro de eventos y snapshots sucesivos del estado. A ello se añaden los artefactos de la parte ejecutable, como el script generado, el payload de entrada, la salida estándar y el registro de error. Gracias a ello, no solo es posible repetir una consulta, sino también reconstruir qué ocurrió exactamente en cada fase cuando se quiere comparar dos ejecuciones o depurar un caso concreto.

Aun así, la reproducibilidad no debe entenderse como identidad garantizada de todas las salidas textuales. La arquitectura y los artefactos pueden repetirse con bastante fidelidad, pero la respuesta final sigue dependiendo de un componente generativo y del contexto de datos disponible en ese momento. Por eso, en este trabajo la reproducibilidad se entiende sobre todo como la capacidad de rehacer el flujo, inspeccionar sus pasos y justificar de qué entradas y de qué evidencias ha salido cada resultado.

Capítulo 8

Conclusiones y trabajo futuro

8.1. Funcion de este capitulo

La conclusion final debe recoger la aportacion principal del trabajo con una formulacion clara y defendible. No conviene que sea una repeticion de capitulos anteriores ni una lista dispersa de ideas futuras.

8.2. Guia para las conclusiones

Completar aqui, al final del proyecto, una sintesis que responda a estas preguntas:

1. ¿Que problema has abordado realmente?
2. ¿Que has conseguido demostrar?
3. ¿Que evidencia es la mas importante para sostener esa afirmacion?
4. ¿Que limite obliga a interpretar el resultado con prudencia?

8.3. Guia para el trabajo futuro

Completar aqui las siguientes lineas de trabajo priorizadas: mejoras del flujo, robustez, validacion, ampliacion de casos, comparativas con otras soluciones y posible evolucion del sistema.

Bibliografía

- [1] Dogu Araci. Finbert: Financial sentiment analysis with pre-trained language models. *arXiv preprint arXiv:1908.10063*, 2019. doi: 10.48550/arXiv.1908.10063. URL <https://arxiv.org/abs/1908.10063>.
- [2] Ran Aroussi. yfinance python library, 2026. URL <https://github.com/ranaroussi/yfinance>.
- [3] Mark Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. doi: 10.48550/arXiv.2107.03374. URL <https://arxiv.org/abs/2107.03374>.
- [4] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*, 2023. doi: 10.48550/arXiv.2304.05128. URL <https://arxiv.org/abs/2304.05128>.
- [5] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. *arXiv preprint arXiv:2211.10435*, 2022. doi: 10.48550/arXiv.2211.10435. URL <https://arxiv.org/abs/2211.10435>.
- [6] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 2022. doi: 10.1145/3571730. URL <https://arxiv.org/abs/2202.03629>.
- [7] Yuke Lai, Chengcheng Li, Yutao Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-tau Yih, Daniel Fried, Sida I. Wang, and Tao Yu. Ds-1000: A natural and reliable benchmark for data science code generation. *arXiv preprint arXiv:2211.11501*, 2022. doi: 10.48550/arXiv.2211.11501. URL <https://arxiv.org/abs/2211.11501>.
- [8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for

- knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2005.11401>.
- [9] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. *arXiv preprint arXiv:2211.09110*, 2022. doi: 10.48550/arXiv.2211.09110. URL <https://arxiv.org/abs/2211.09110>.
 - [10] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chengguang Zhu. G-eval: Nlg evaluation using gpt-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2511–2522. Association for Computational Linguistics, 2023. doi: 10.18653/v1/2023.emnlp-main.153. URL <https://aclanthology.org/2023.emnlp-main.153/>.
 - [11] OpenAI. gpt-oss-120b & gpt-oss-20b model card, 2025. URL <https://arxiv.org/abs/2508.10925>.
 - [12] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. Asleep at the keyboard? assessing the security of github copilot’s code contributions. In *2022 IEEE Symposium on Security and Privacy*, pages 754–768. IEEE, 2022. doi: 10.1109/SP46214.2022.9833571. URL <https://arxiv.org/abs/2108.09293>.
 - [13] Ji-Lun Peng, Sijia Cheng, Egil Diau, Yung-Yu Shih, Po-Heng Chen, Yen-Ting Lin, and Yun-Nung Chen. A survey of useful llm evaluation. *arXiv preprint arXiv:2406.00936*, 2024. doi: 10.48550/arXiv.2406.00936. URL <https://arxiv.org/abs/2406.00936>.
 - [14] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023. doi: 10.48550/arXiv.2302.04761. URL <https://arxiv.org/abs/2302.04761>.
 - [15] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *arXiv preprint arXiv:2308.11432*, 2023. doi: 10.48550/arXiv.2308.11432. URL <https://arxiv.org/abs/2308.11432>.
 - [16] Shijie Wu, Ozan Irsoy, Steven Lu, Vadim Dabravolski, Mark Dredze, Sebastian Gehrmann, Prabhanjan Kambadur, David Rosenberg, and Gideon Mann. Bloomberggpt: A large language model for finance. *arXiv preprint arXiv:2303.17564*,

2023. doi: 10.48550/arXiv.2303.17564. URL <https://arxiv.org/abs/2303.17564>.
- [17] Yijia Xiao, Edward Sun, Di Luo, and Wei Wang. Tradingagents: Multi-agents llm financial trading framework. *arXiv preprint arXiv:2412.20138*, 2024. doi: 10.48550/arXiv.2412.20138. URL <https://arxiv.org/abs/2412.20138>.
- [18] Yahoo Finance Help. Download historical data in yahoo finance, 2026. URL <https://help.yahoo.com/kb/finance/historical-prices-adjusted-close-sln2311.html>.
- [19] Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. Fingpt: Open-source financial large language models. *arXiv preprint arXiv:2306.06031*, 2023. doi: 10.48550/arXiv.2306.06031. URL <https://arxiv.org/abs/2306.06031>.
- [20] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- [21] Tianyu Zhou, Pingqiao Wang, Yilin Wu, and Hongyang Yang. Finrobot: Ai agent for equity research and valuation with large language models. *arXiv preprint arXiv:2411.08804*, 2024. doi: 10.48550/arXiv.2411.08804. URL <https://arxiv.org/abs/2411.08804>.

Apéndice A

Anexos

A.1. Ejecucion del sistema

Este anexo recoge la informacion operativa necesaria para ejecutar y defender el proyecto. El punto de entrada principal es `run_mvp.py`, que recibe una consulta en lenguaje natural, construye el workflow multiagente y devuelve un JSON final con el estado completo de la ejecucion.

A.1.1. Preparacion minima

Antes de lanzar una ejecucion conviene comprobar cuatro condiciones:

1. que el entorno Python tenga instaladas las dependencias del proyecto;
2. que el fichero `.env` contenga una configuracion LLM valida;
3. que el entorno tenga conectividad suficiente para acceder tanto al endpoint LLM como a `yfinance`;
4. que el proceso pueda escribir los artefactos generados en la carpeta `results`.

Una instalacion minima puede hacerse con:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

A.1.2. Ejecucion con ejemplos integrados

El proyecto incluye varios casos ya preparados en `src/examples/sample_inputs.py`. Estos ejemplos sirven para lanzar consultas representativas sin tener que redactarlas manualmente y permiten comprobar el flujo de extremo a extremo con peticiones ya

definidas dentro del repositorio. La forma mas simple de probar el sistema es ejecutar uno de esos ejemplos:

```
python run_mvp.py --example growth_nvda
python run_mvp.py --example returns_qqq_spy
python run_mvp.py --example complex_nvda_growth_report
```

Esta via es la mas util para una primera validacion, porque evita errores de formato en la entrada y permite observar como el sistema resuelve la peticion, descarga los datos necesarios y genera la respuesta final.

A.1.3. Ejecucion con una entrada JSON propia

Tambien es posible lanzar el sistema con un fichero JSON preparado manualmente:

```
python run_mvp.py --input-json ruta\al\caso.json
```

La estructura minima esperada es la siguiente:

```
{
  "query": "Analiza los retornos de QQQ y SPY en 2024"
}
```

La entrada minima solo necesita el campo **query**. A partir de esa peticion, el workflow resuelve los instrumentos, el rango temporal y los parametros de descarga antes de pasar a la fase analitica. La ejecucion normal del sistema no depende de proporcionar rutas de CSV locales en la entrada.

A.1.4. Salida y artefactos generados

La ejecucion imprime por consola un JSON con el estado final del workflow. Ese estado incluye, entre otros elementos:

- la consulta original y su forma normalizada;
- el plan analitico generado por el agente planificador;
- el codigo Python producido para el calculo;
- la salida estructurada de la ejecucion;
- la respuesta final redactada para el usuario;
- el estado de la ejecucion, los avisos y los posibles errores.

Ademas, el sistema conserva artefactos intermedios en dos carpetas:

- `results/data_raw` y `results/data_normalized`, con los artefactos generados durante la fase de descarga y normalizacion de datos;
- `results/code`, con el codigo generado en cada ejecucion;
- `results/logs`, con el payload enviado al ejecutor y los ficheros de `stdout` y `stderr`.

Esta conservacion resulta util tanto para depurar fallos como para defender metodologicamente la trazabilidad del flujo en la memoria.

A.2. Configuracion LLM y API

La aplicacion utiliza un endpoint compatible con OpenAI Chat Completions. El modelo `openai/gpt-oss-20b` [11] se sirve a traves de vLLM, aunque el codigo acepta varias familias de variables de entorno equivalentes.

A.2.1. Configuracion recomendada

La forma mas directa de configurar el sistema es declarar en `.env`:

```
LLM_PROVIDER=openai-compatible
VLLM_BASE_URL=https://servidor-vllm.example/v1
VLLM_API_KEY=pegar_aqui_la_clave
VLLM_MODEL=openai/gpt-oss-20b
```

A.2.2. Variables equivalentes admitidas

El codigo reconoce tres familias de nombres para base URL, clave y modelo:

- familia `VLLM_*`: `VLLM_BASE_URL`, `VLLM_API_KEY`, `VLLM_MODEL`;
- familia `LLM_*`: `LLM_BASE_URL`, `LLM_API_KEY`, `LLM_MODEL`;
- familia `OPENAI_*`: `OPENAI_BASE_URL`, `OPENAI_API_KEY`, `OPENAI_MODEL`.

La resolucion de configuracion prioriza primero los valores `VLLM_*`, despues los valores genericos `LLM_*` y finalmente los alias `OPENAI_*`.

A.2.3. Parametros de generacion

Tambien pueden ajustarse los parametros principales del cliente:

```
LLM_TEMPERATURE=0.1  
LLM_MAX_TOKENS=4096  
LLM_TIMEOUT_SECONDS=120  
OPENAI_VERIFY_SSL=true
```

Estos valores controlan la temperatura de generacion, el maximo de tokens de salida, el tiempo de espera del cliente y la verificacion SSL del endpoint remoto.

A.2.4. Comprobaciones practicas

Si la configuracion es correcta, una ejecucion con `run_mvp.py` debe completar el flujo resolviendo la consulta, descargando los datos historicos necesarios y generando despues una respuesta trazable. Si la clave no esta definida o contiene un valor de ejemplo, el sistema no intenta llamar al modelo y devuelve un error de configuracion explicito.

A.3. Notas para reproducibilidad

Este anexo sirve como referencia corta de reproduccion operativa. Para repetir un caso hacen falta el mismo codigo, una configuracion equivalente del modelo, acceso a un endpoint compatible con OpenAI y disponibilidad de los datos historicos correspondientes en `yfinance`. Si alguno de estos elementos cambia, el comportamiento del sistema puede variar aunque la arquitectura general siga siendo la misma.